MASTER'S THESIS · COLLOQUIUM · OCTOBER 2026

Behavior of Python Static Type Checkers under Domain-Model Changes in ORM-Based Projects

An ORM-aware extension

Ilnar Khasanov

PROBLEM

One model change, many places to edit

The change: extract an Address entity

```
class Order(Base):
- delivery_address: Mapped[str]
+ address: Mapped[Address] = relationship()
```

Extract Entity is listed as an open challenge problem of schema evolution (Edwards et al., 2025).

Where the old field is still used

`order.delivery_address.upper()`	Reported
`fields = ["delivery_address"]`	Missed
`.filter(delivery_address=x)`	Missed
`"SELECT delivery_address …"`	Missed
`getattr(order, field_name)`	Missed
`fixtures/orders.json`	Missed

PROBLEM

Teams use mypy and pyright to find what a model change breaks. How much of the real impact they see has never been measured.

Dynamic ORM attributes

Relations and reverse accessors are generated at runtime; ORM plugins for mypy cover them only partly.

Fields named by strings

Serializers, query filters, admin, forms and templates refer to fields as plain text.

Code outside Python types

Raw SQL, migrations and JSON fixtures sit outside the type system entirely.

Motivation: an industrial Python back-end with 1M+ lines and mypy in CI, where adding one attribute triggers a cascade of edits.

LITERATURE

Four strands of prior work

Strand	Key works	What they establish	Limit for this thesis
Schema–code co-evolution	Qiu et al., FSE 2013; Chen et al., MSR 2016	Schema changes drive many code edits; commits as ground truth	Java, PHP; no type checker
Schema-change impact analysis	SUITE (Maule et al., ICSE 2008); Meurice et al., QRS 2016	Predict affected code locations before the change	C#, Java; never compared with a type checker
Evaluating impact analysis, LLMs	Gethers et al., ICSE 2012; Athena, FSE 2024; Ripple, ICSE 2026	Precision and recall against commits; best LLM: P 28%, R 36%	Java only
Python type checkers	Khan et al., TSE 2021; Rak-amnouykit et al., DLS 2020; PyTy, ICSE 2024	~15% of defects preventable by mypy; checkers flag different errors	Defects and repair, not change impact

Also: Edwards et al. (2025) on schema evolution; Ziftci et al. (2025), LLM-assisted migrations at Google, where finding the change locations is a separate step.

LITERATURE

Three gaps this thesis addresses

01

No measurement

No study measures what share of a model change's real impact Python type checkers report.

02

No Python / ORM evaluation

Schema-impact tools were built for C# and Java, and none compares itself with a type checker.

03

LLMs untested here

The best published LLM impact analysis reaches 36% recall on Java bug fixes; ORM-based Python is unexplored.

APPROACH

Research questions

RQ1 · Measure

How much of the impact of domain-model changes do type checkers detect (recall, precision), by checker, configuration and change type?

RQ2 · Explain

Which categories of impacted locations do type checkers miss, and why?

RQ3 · Improve

How much does an ORM-aware extension add, at what cost in precision, compared with co-change and dependency baselines?

RQ4 · Compare

How do LLM-based detectors compare with type checkers and the extension, and do they complement them?

APPROACH

Study design: replay real model changes

1	**Mine history**	Open-source Django and SQLAlchemy projects; industrial back-end if permitted
2	**Detect model changes**	AST diff of model classes → add, remove, rename, retype, extract entity
3	**Replay the change**	Check out the parent revision and apply only the model hunks of the commit
4	**Run the checkers**	mypy and pyright; errors that appear after the change = predicted impact
5	**Compare**	Predicted locations vs. files and functions the developers actually changed

RQ1 → recall and precision per checker, configuration and change type

RQ2 → missed locations classified by hand into a taxonomy of blind spots

APPROACH

The improved tool: an ORM-aware layer

L1	**Type-checker errors** — the baseline the extension must beat	Mandatory
L2	**ORM-aware references** — field names in strings: filters, serializers, admin	Mandatory
L3	**SQL and data artifacts** — raw SQL, migrations, fixtures (parsed with sqlglot)	Planned
L4	**Co-change ranking** — files that historically change together with the model	Planned
L5	**Dynamic tracing** — attribute access recorded while the test suite runs	If time allows

Output: an impact report where every location shows its evidence (type error, string reference, SQL, co-change).

Scope: a layer on top of mypy and pyright, not a change to the checkers; each layer's gain is measured separately.

Evaluation: dataset and ground truth

Subjects

Open-source Django and SQLAlchemy projects with model-changing commits and a runnable checker setup.

Ground truth

Files and functions changed in the same commit, excluding model definitions and migrations (protocol of Qiu et al., 2013).

Noise control

Tangled commits filtered by heuristics; at least 10% of commits checked by hand to estimate ground-truth precision.

ILLUSTRATIVE EXAMPLE

`retype Order.total: int → Decimal`

Changed by developers (4):

`api/views.py`	**Found**
`api/serializers.py`	**Missed**
`reports/orders.sql`	**Missed**
`tests/test_orders.py`	**Missed**
`admin.py (reported only)`	**Extra**

Recall 1/4 = 25% · Precision 1/2 = 50%

Evaluation: experiment and metrics

Detectors, same commits

- mypy and pyright: project, default and strict configs
- Extension, layer by layer (L1 → L4)
- Baselines: co-change mining, dependency closure, LLM agent (pinned model, 3 runs)

Metrics

- Precision, recall, F1 at file and function level
- Broken down by checker, config and change type
- Macro and micro averages, also per project

Statistics

- Paired Wilcoxon signed-rank test per commit
- Cliff's δ effect size, bootstrap confidence intervals

Success criterion, fixed in advance: recall significantly higher than the best single checker, with precision ≥ 0.8. A null result still answers RQ1: how far type checkers can be trusted.

WHAT IS NEW COMPARED WITH OTHER APPROACHES?

Novelty: the closest work, side by side

Approach	Stack	vs. type checkers	ORM-aware	Evaluated on
SUITE, ICSE 2008	C#, embedded SQL	No	No	One system, 62 schema versions
Meurice et al., 2016	Java systems*	No	Yes	History of real systems
Oopsie, arXiv 2026	Java, JDBC	Is a checker, no impact	No	Test suite, one real bug
Ripple, ICSE 2026	Java, any change	No	No	100 bug-fix commits
This thesis	Python: Django, SQLAlchemy	Yes: mypy, pyright	Yes	Model-changing commits, hand-checked sample

Measurement: first recall and precision of type checkers as impact detectors

Taxonomy: what checkers miss in ORM code, and why

Tool and data: open ORM-aware extension and dataset

PLAN

Plan and risks

Done	Topic, literature review, proposal
Nov	Pilot on 3 projects, first recall estimate
Dec	Dataset at scale, manual validation
Jan–Feb	Checker runs (RQ1), taxonomy (RQ2), L2
Mar	L3–L4, ablation, classical baselines (RQ3)
Apr	LLM experiments (RQ4), packaging
May	Writing and defence

Risk	Mitigation
Noisy ground truth	Tangle filter, hand-checked sample
Too few model changes	Relax criteria, add projects or injected changes
LLM saw the code in training	Split by training cutoff; private repo as clean check
No permission for work repo	Open-source data only
Checkers see almost all	Still a publishable answer to RQ1

SUMMARY

Thank you — questions?